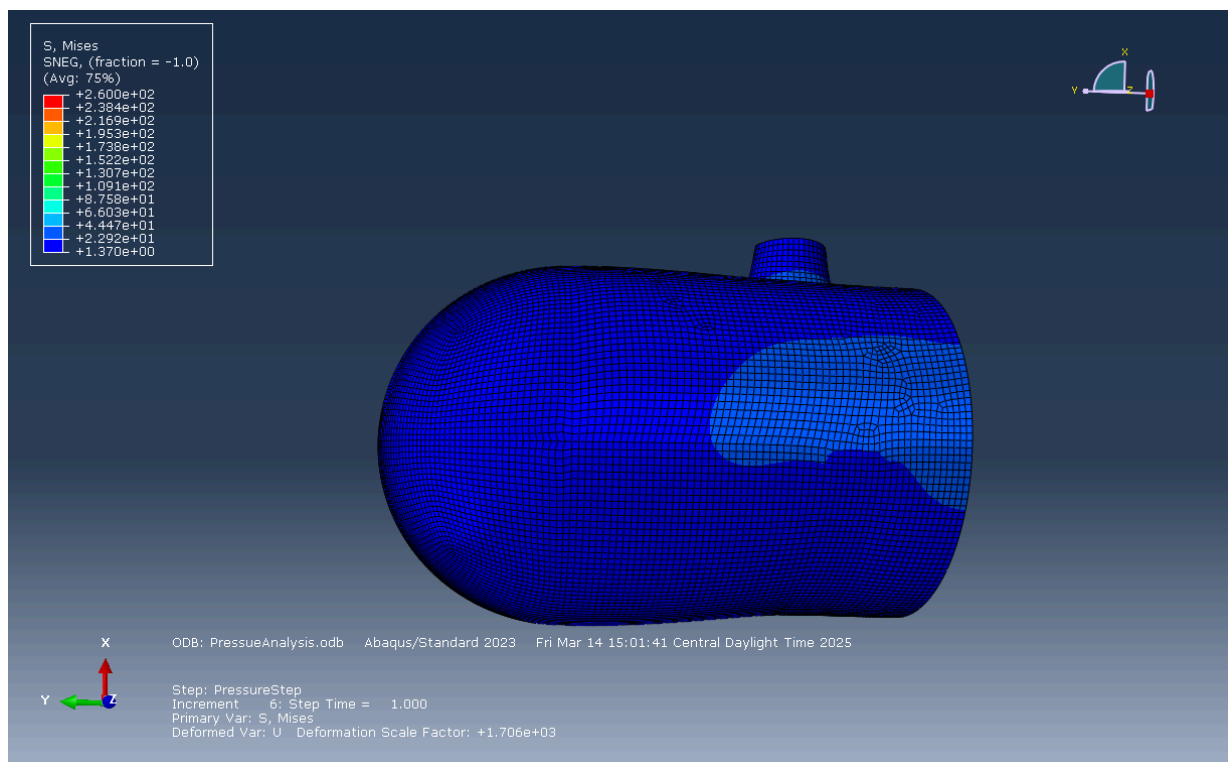


Optimization of a Pressure Vessel

Using Abaqus FEA

ME 341 Winter 2025



Alexander Torres-Soto, Xing Yu Chen
McCormick School of Engineering & Applied Science, Northwestern University

Table of Contents

Introduction.....	3
Problem Statement.....	3
Why Optimization is Needed?.....	4
Objective of the Project.....	4
Technical Background.....	5
Physical Representation.....	5
Optimization Algorithm.....	6
Optimization Formulation.....	7
Mathematical Formulation.....	7
Design Variables:.....	7
Objective Function:.....	7
Constraints:.....	8
Penalty Function for Constraints:.....	9
Finite Element Analysis (FEA) in Abaqus.....	9
Analysis of Results.....	12
Software.....	12
Attempting Convergence.....	12
Verification.....	16
Numerical Implementation.....	16
Random Sampling Comparison.....	17
Validation.....	18
Reality Check.....	18
Volumetric Validation.....	18
Mass Validation.....	19
K-Fold Cross Validation.....	20
Conclusion & Future Plans.....	21
Conclusion.....	21
Future Work.....	22
Achieved Learning.....	23
Alexander Torres-Soto.....	23
Xing Yu Chen.....	23
References.....	24
Appendix.....	25

Introduction

Problem Statement

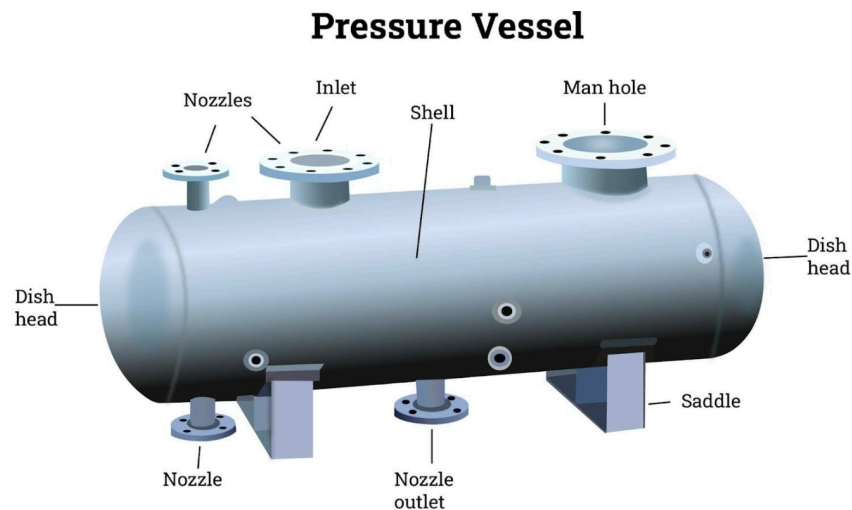


Figure 1. Horizontal Pressure Vessel Overview

Pressure vessels are widely used in industries such as chemical processing, power generation, aerospace, and manufacturing. These vessels are designed to contain fluids under pressure while ensuring structural integrity and safety. However, designing an optimal pressure vessel presents several engineering challenges. The primary concerns include minimizing the mass of the vessel while ensuring it meets strength, volume, and regulatory requirements.

In engineering applications, excessive material usage leads to higher manufacturing costs, increased weight, and inefficiencies in operation. Conversely, reducing material usage without proper optimization may lead to structural failure due to stress concentration, buckling, or exceeding allowable pressure limits.

Given these challenges, a systematic optimization approach is required to identify the most efficient design that balances material usage, structural integrity, and compliance with safety standards [1].

Why Optimization is Needed?

The traditional method of designing pressure vessels relies on empirical formulas and conservative safety factors, often leading to over-engineered and unnecessarily heavy structures. While these designs are safe, they lack efficiency and cost-effectiveness. With modern computational tools, optimization techniques allow engineers to achieve lighter, stronger, and more cost-effective designs by systematically evaluating various design parameters [2].

In this project, optimization is essential for:

1. **Minimizing Material Usage:** Reducing the vessel's weight lowers manufacturing and transportation costs.
2. **Ensuring Structural Safety:** The vessel must withstand internal pressure without exceeding stress limits defined by design codes.
3. **Automating the Design Process:** Rather than relying on trial-and-error, an optimization framework provides a systematic method for achieving an optimal design [3].

Objective of the Project

The objective of this project is to develop an optimization framework for designing a pressure vessel that minimizes mass while satisfying geometric, material, and stress constraints.

Specifically, the project aims to:

- **Formulate the optimization problem** by defining design variables, constraints, and an objective function.
- **Implement Bayesian Optimization** to iteratively refine the design by leveraging Gaussian Process models for efficient search in the design space.
- **Integrate Abaqus Finite Element Analysis (FEA)** to validate the maximum stress within the vessel and ensure it remains within allowable limits.
- **Compare different optimization approaches** by evaluating results with varying initial conditions and penalty multipliers.
- **Develop a penalty function** to handle constraint violations continuously rather than using hard constraints

Technical Background

Physical Representation

To understand the physics behind pressure vessels, let us first establish some conditions:

- Cylindrical in Geometry (most common)
- Internal pressure (P) is uniformly distributed across the inner surface
- Material properties of Carbon Steel (SA-516 GR.70) are summarized in table 1[4]:

Table 1: Material Properties

Material: SA-516 GR.70	
Density	7750 Kg/m ³
Modulus of Elasticity	193 GPa
Poisson ratio	0.3
Yield Strength (σ_{allow})	260 MPa
Ultimate tensile strength	493 MPa

Pressure vessels are classified as thin-walled or thick-walled problems. In the thin walled case, the wall thickness (t) is much smaller than the inner radius (r) (in our case, by a factor of 10). We can define the hoop stress (σ_{θ}) (stress tangential to the circumference of the cylinder) and the axial stress (σ_a) (stress along the longitudinal axis of the cylinder) as:

$$\sigma_{\theta} = \frac{pr}{t}, \sigma_a = \frac{pr}{2t} \quad (1)$$

The radial stress (σ_r) is negligible for thin-walled pressure vessels ($\sigma_r = 0$).

For thick-walled pressure vessels (where $t \gg r$):

$$\sigma_{\theta} = \frac{b}{r} + a, \sigma_a = \frac{b}{r^2} - a \quad (2)$$

Where a and b are constants set by the boundary conditions of the pressure vessel [6]. Eq. (2) is an oversimplification of Lamé's equations involving thick-walled vessels, as we will primarily be focusing on the thin-walled case for our project.

Optimization Algorithm

We chose to use Bayesian Optimization over other algorithms because it can handle black-box functions (i.e. our model's stress concentration, requiring FEA) efficiently, which reduces the number of evaluations needed for an optimal solution.

Bayesian Optimization comprises of two components:

- 1) Surrogate Model: Gaussian Process (GP)
- 2) Acquisition Function: Expected Improvement (EI)

The surrogate model approximates the objective function and identifies regions of uncertainty. The Gaussian process offers a probabilistic model of the objective function based on observed data, characterized by a mean function (representing the expected value at any given point) and a covariance function (indicating the correlation between different points).

The acquisition function determines the next point to evaluate by balancing exploration (sampling areas with the highest uncertainty) and exploitation (sampling where the surrogate model predicts the best outcomes). Expected Improvement takes into account the results from previous iterations, including the current best result, to guide the search towards promising regions. This approach ensures that the algorithm continuously refines its search and improves performance with each iteration.

We utilized Python's built-in function, `gp_minimize`, to optimize our design due to its efficient and flexible gaussian process surrogate model, which is well-suited for solving complex optimization problems. The algorithm initially gathers samples during the first few iterations and then continues optimizing by using Expected Improvement (EI) as the acquisition function.

Optimization Formulation

Mathematical Formulation

The optimization problem for the pressure vessel design is formulated as follows:

Design Variables:

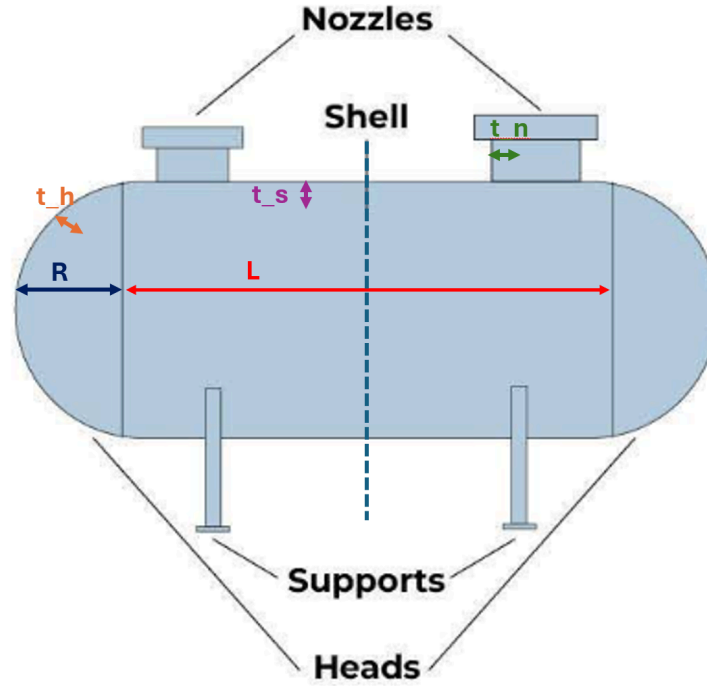


Figure 2. Design variables of optimization problem

The design variables represent the key geometric parameters that affect the vessel's performance as shown in Figure 2 above:

- R : Shell outer radius (mm)
- L : Shell length (mm)
- t_s : Shell thickness (mm)
- t_h : Head thickness (mm)
- t_n : Nozzle thickness (mm)

Objective Function:

The objective is to **minimize the total mass (tonnes, 1 tonne = 1000 kg) of the pressure vessel** while ensuring it meets structural and volume constraints. The mass is computed as:

$$M(R, L, t_s, t_h, t_n) = \rho \times V_{total} + P(X) \quad (3)$$

where:

- $P(X)$ is the penalty function
- ρ is the material density (tonne/mm³)
- V_{total} is the total volume of the vessel:

$$V_{shell} = 2\pi R L t_s, V_{heads} = 4\pi R^2 t_h, V_{nozzle} = 2\pi R_n H_n t_n, V_{hole} = 2\pi R_n t_n \quad (4)$$

$$V_{total} = V_{shell} + V_{heads} + V_{nozzle} - V_{hole} \quad (5)$$

$$R_n = H_n = 150 \text{ mm}$$

Constraints:

Several constraints must be satisfied to ensure a feasible design:

1. Minimum Volume Constraint:

$$\pi R^2 L + \frac{3}{4}\pi R^3 \geq V_{req} \quad (6)$$

Where: $V_{req} = 2.271\text{e}+8 \text{ (mm}^3\text{)}$

2. Minimum Thickness Constraints (Strength Condition):

$$t_s \geq \frac{P \times R}{2\sigma_{allow}} \quad (7)$$

$$t_h \geq \frac{P \times R}{2\sigma_{allow}} \quad (8)$$

$$t_n \geq \frac{P \times R}{2\sigma_{allow}} \quad (9)$$

3. Stress Constraint (from Abaqus FEA):

$$\sigma_{max} \leq \frac{\sigma_{allow}}{3} \quad (10)$$

Geometric boundaries:

Shell length (L): 1200 - 5000 mm

End Cap radius (R): 300 - 2000 mm

Shell thickness (t_s): 40 - 500 mm

End Cap thickness (t_h): 40 - 500 mm

Nozzle thickness (t_n): 30 - 100 mm

Penalty Function for Constraints:

Instead of rejecting infeasible designs, constraint violations are penalized using a continuous function:

$$P(X) = \sum_{i=1}^n \lambda_i g_i^2(X) \quad (11)$$

where:

- $g_i(X)$ is the constraint violation.
- λ_i are penalty multipliers determined through parameter tuning.

Finite Element Analysis (FEA) in Abaqus

The stress constraint number mentioned in the above section is evaluated using **Abaqus CAE 2023 Finite Element Analysis (FEA)** at each iteration of the optimization to ensure that the maximum stress in the vessel remains within allowable limits. The key steps in the simulation are

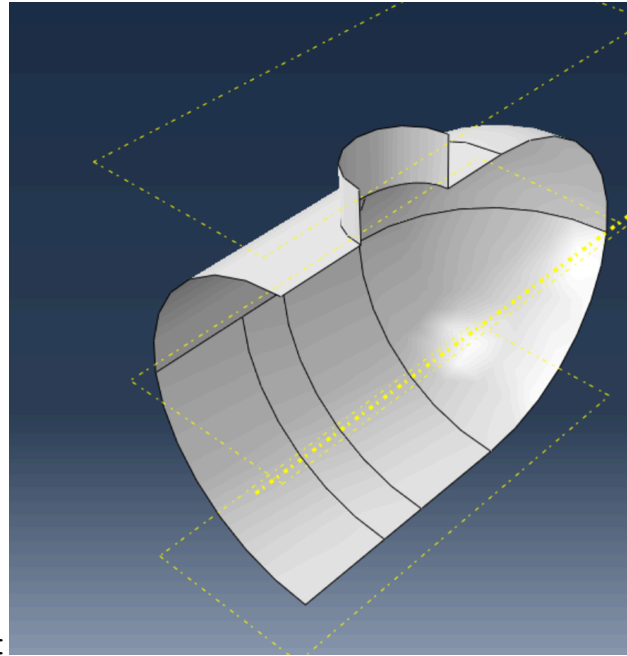


Figure 3a. One quadrant of a pressure vessel design

1. **Input Parameters:** The geometric dimensions are provided to Abaqus via a Python script. The code then models a quadrant of the pressure vessel to minimize the computational time in resources during the optimization iterations as shown in Figure 3a.

2. **Meshing and Material Properties:** The model is meshed using shell elements, and the material is described in the technical background section. The thickness for each part is applied using shell thickness on Abaqus.

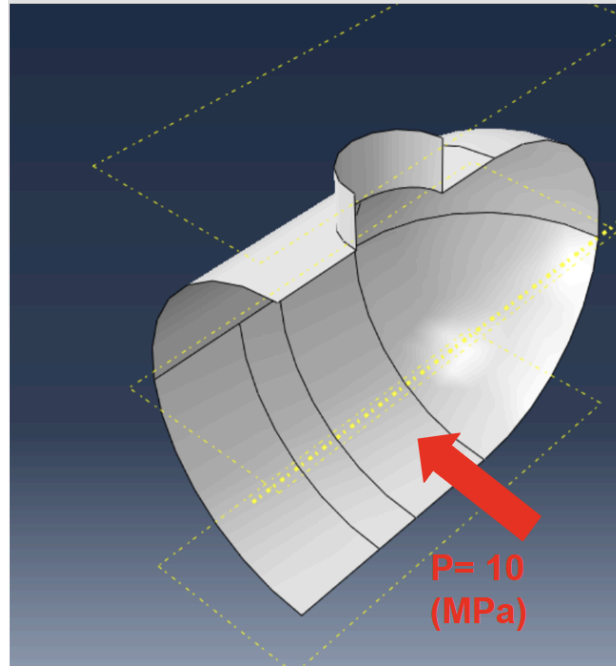


Figure 3b. Loading conditions on inner surfaces

3. **Loading Conditions:** The pressure of 10 MPa was applied to the inner surface of the pressure vessel, highlighted in pink arrows in the figure 3b above.

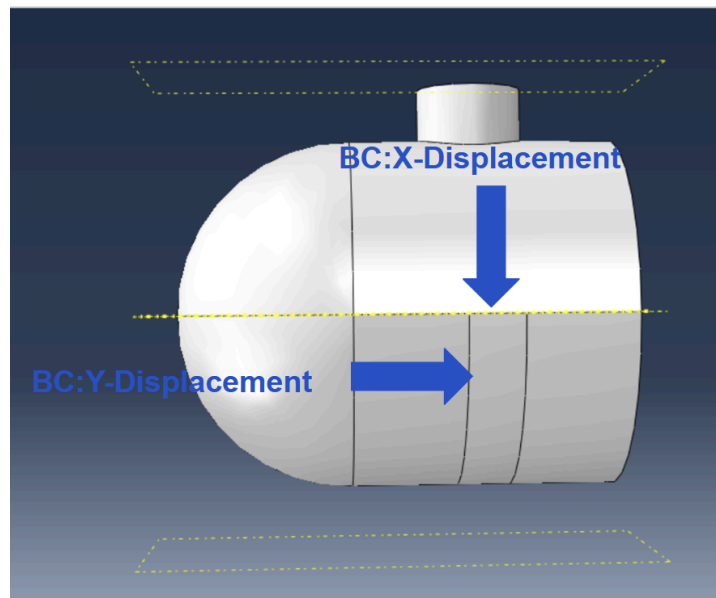


Figure 3c. Saddle boundary conditions

4. **Boundary Conditions:** Created symmetrical boundary conditions at the edges of the quadrant, in the Y-axis and Z-axis. Created displacement constraints on the rectangle at the bottoms of the shell as seen in Figure 3c, this is meant to simulate the saddle support of the pressure vessel. The constraints prevent the Y and X-axis displacement
5. **Solution and Post-Processing:** Abaqus computes stress distributions, and the maximum stress is extracted from the results file. The solution for the initial design took around 10 seconds, however this time increases as R and L increase as more elements are created.
6. **Stress Validation:** If the stress constraint is violated, a penalty is applied to guide the optimizer away from infeasible regions.

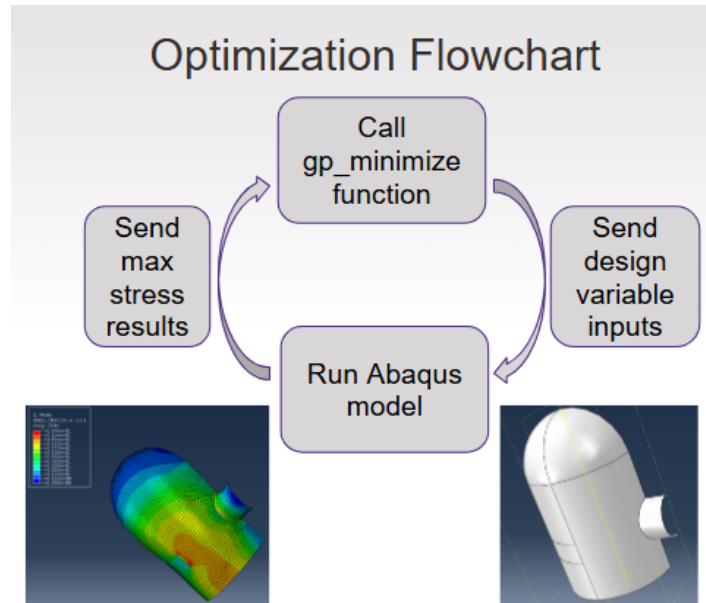


Figure 4. Optimization Flowchart

The figure above showcases the optimization flowchart for every iteration. We call our main code Bayesian_Optimization.py which computes the mass and constraints at each iteration while using the function gp_minimize for optimization. The main code sends the iteration's design variables to another code called runAbaqus.py which creates an Abaqus FEA simulation and extracts the maximum stress of the design. Then sends the max stress to the main code to check if the stress constraint is violated and a penalty is applied to guide the optimizer away from infeasible regions.

Analysis of Results

Software

We used Python/MATLAB to make our plots and called Abaqus to generate the FEA figures.

Attempting Convergence

For our first set of runs, we started with an initial sample size of 30 and added 30 additional iterations. We performed this for three different starting points (x1, x2, x3) as shown in the table below to observe the convergence of different starting points:

Table 2: Initial Positions

Starting Point	R (mm)	L (mm)	t_s (mm)	t_h (mm)	t_n (mm)
X1	500	1500	60	60	65
X2	300	1300	100	100	60
X3	350	1300	70	70	75

The following figures show the objective vs iteration for the three different starting points, to clarify we are minimizing a continuous penalty function in addition to the mass this helps us simplify our optimization formulation as explained in the previous section. The spikes in the objective as seen in Figure 5, are due to the iteration design violating a constraint thus the objective spikes up and in the next iteration the algorithm searches in another direction.

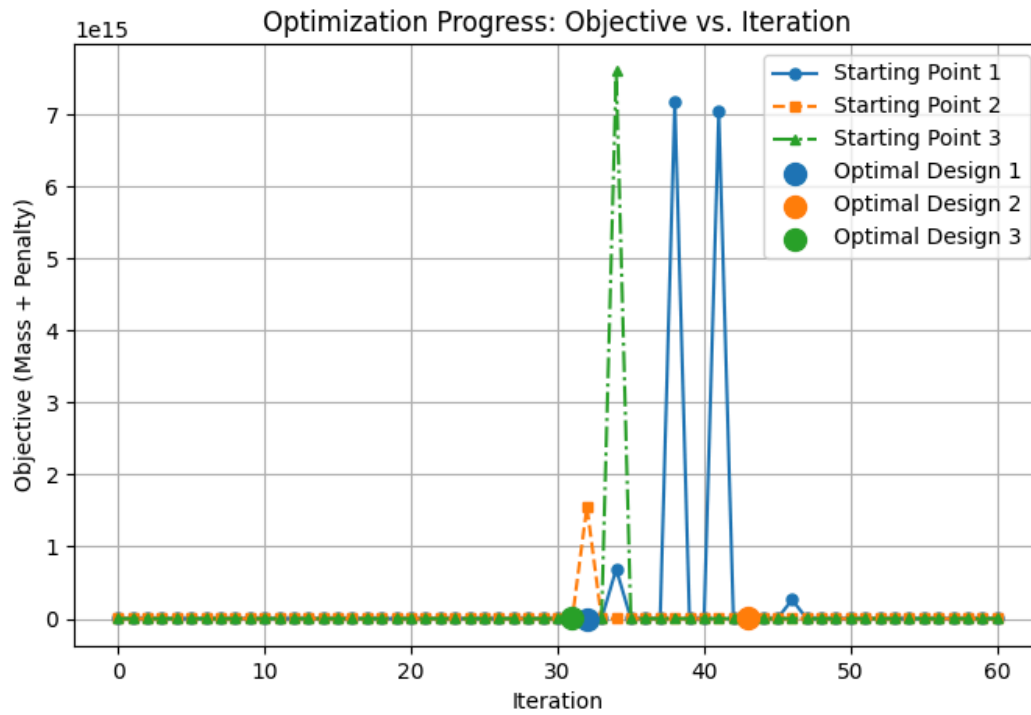


Figure 5. *Objective vs Iterations for different starting points*

Figures 6 and 7 show the optimization history of the mass without penalty and the max stress from Abaqus, marking the optimal solution for each starting point. The plots show that the optimal objective has a minimum mass and no penalties are violated, as the max stress is constrained at a value of 86.66 MPa. As observed in Figure 7, at some iterations there are masses lower than the optimal values, this means that the iteration's design violates the constraints at those masses thus not making it an optimal point.

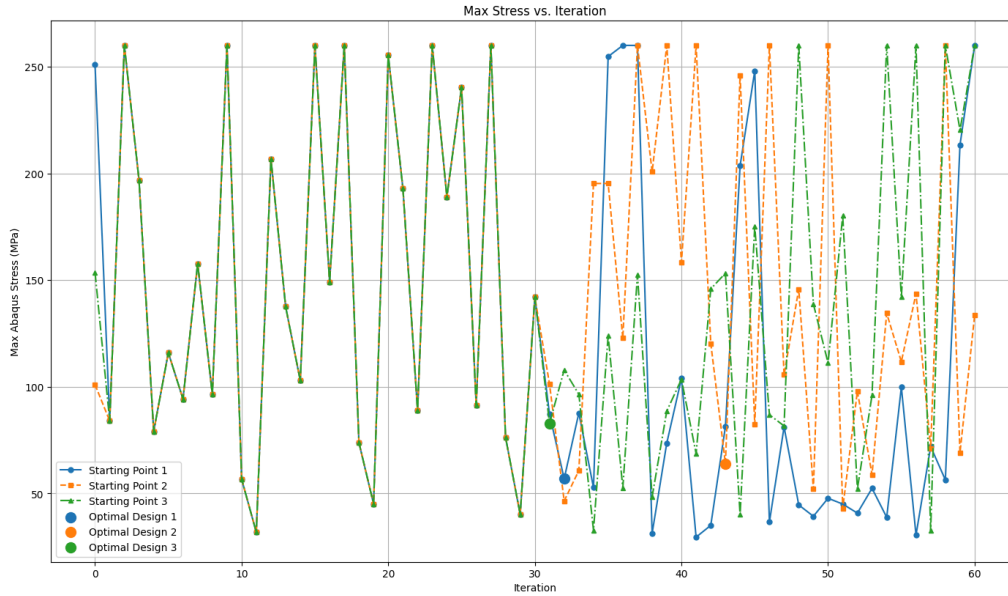


Figure 6. Max Stress vs Iterations for different starting points

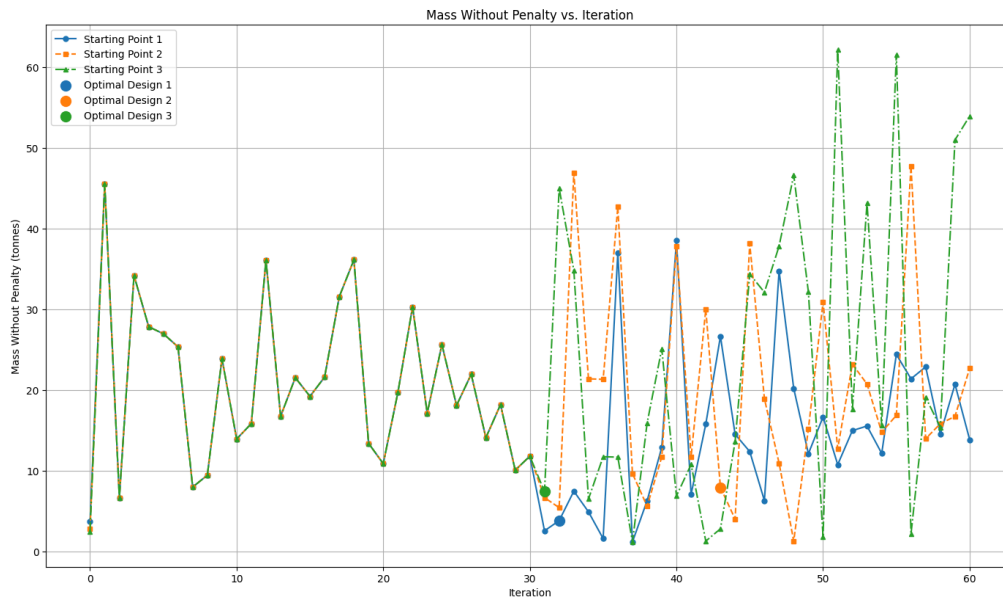


Figure 7. Mass vs Iterations for different starting points

To explain why the different starting points don't converge to the same optimal value we need to understand the optimization algorithm. BO balances exploration (searching new areas) and exploitation (refining known good areas) using an acquisition function. Different starting points cause different exploration-exploitation patterns, leading to different final solutions. Unlike

Genetic Algorithms or Differential Evolution, BO does not guarantee convergence to the global optimum, thus explaining why there are different optimal designs based on the starting points. Next, we tested the algorithm's efficiency by running 10 total iterations (5 initial samples and 5 additional iterations) to see if it could produce reliable results with fewer iterations.

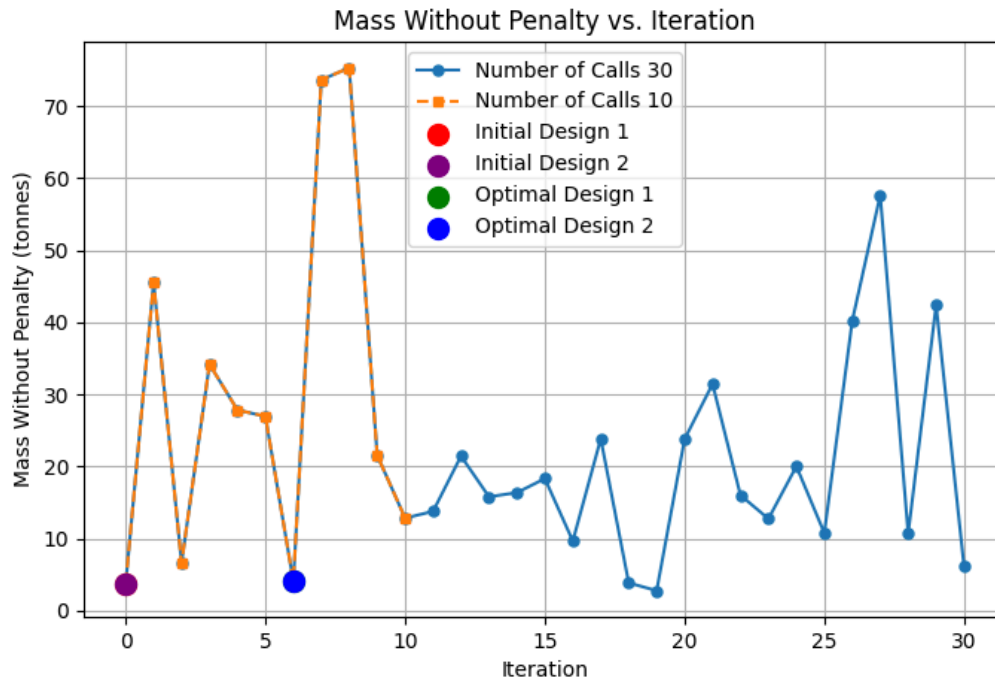


Figure 8. Mass Plot of 10 vs. 30 iterations

For the second set of runs, used the same number of samples (10 initial samples) and the same starting point with two different numbers of call 10 and 30 iterations. The figure above shows that even with different numbers of calls with the same initial point the problem would converge to the same point. This helps us assess our performance as there is no need for a greater number of calls for each starting point as we know it will converge to the same point. The overall time for 30 initial samples and 30 calls was 27 minutes, which will increase if there is a greater number of calls. The tradeoffs of this method is that since we are using Abaqus simulation as a black box problem there is no global minimum that we can achieve thus the starting point affects the optimal value.

Verification

Numerical Implementation

To verify the accuracy of our equations, our code writes the results of each iteration into "optimization_data.csv" after Abaqus completes a run. The recorded data includes the mass calculated using Eq. 3 and the mass computed by Abaqus for our part. Since our Abaqus model represents a quarter of the entire vessel, the Abaqus mass should always be one-fourth of the calculated mass, which we have verified. Additionally, we print the deficit values for each constraint to identify any violated constraints.

The figure below represents how the data collected from the optimization runs is saved.

Table 3: Data Verification

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	R	L	t_s	t_h	t_n	MaxStress	MassWithc	MassFrom	Objective	VolumeDe	t_s_Deficit	t_h_Deficit	t_n_Deficit	StressExcess	
2	500	1500	60	60	65	251.0528	3.6904485	0.934738	270231.68	0	0	0	0	164.38612833333332	
3	714.9912	3563.9440	263.06586	260.28545	73.649458	84.01173	45.537392	11.37026	45.537392	0	0	0	0	0	
4	569.0672	2033.0968	54.745374	110.89063	63.436558	260	6.621075	1.667354	300451.06	0	0	0	0	173.33333333333331	
5	868.51811	2543.9360	142.12404	257.38047	53.617731	196.8158	34.179799	8.541338	121362.45	0	0	0	0	110.14911433333332	
6	753.72031	2231.0763	288.86034	76.491202	90.906108	79.1118	27.826766	6.944121	27.826766	0	0	0	0	0	
7	631.52563	3442.5501	175.32414	216.50867	80.444285	115.9681	26.962274	6.740921	8612.7075	0	0	0	0	29.301442333333327	
8	707.4139	2704.6450	237.24006	67.535977	63.152029	94.25058	25.333840	6.319741	600.49125	0	0	0	0	7.583913333333328	
9	430.43264	3263.3708	96.303092	75.156725	52.689870	157.7461	7.9481539	1.990448	50530.840	0	0	0	0	71.07945733333334	
10	404.77240	1822.4998	140.48713	274.67560	61.496499	96.4626	9.4198073	2.355067	969.02310	0	0	0	0	9.795934333333335	
11	729.14442	3726.5760	65.81289	292.15035	75.719802	260.0001	23.881592	5.984233	300468.64	0	0	0	0	173.3334253333333	
12	419.63670	2202.8260	235.17839	198.03597	52.753306	56.67869	13.911256	3.461604	13.911256	0	0	0	0	0	
13	326.89779	2975.9673	289.32680	209.72548	74.454121	31.81686	15.811729	3.936299	15.811729	0	0	0	0	0	
14	996.70969	2829.1809	147.73583	163.42135	73.645707	206.6958	36.096716	9.025567	144106.13	0	0	0	0	120.02918033333331	
15	536.60533	3089.3065	122.4725	242.36982	96.469973	137.5754	16.721683	4.193518	25933.752	0	0	0	0	50.90877233333332	
16	763.76880	1238.0005	201.93998	215.15150	98.036150	102.8748	21.517779	5.381039	2648.5617	0	0	0	0	16.20815833333333	
17	914.7354	2626.9482	54.485820	157.30139	31.399136	260.0001	19.198460	4.801442	300463.96	0	0	0	0	173.3334253333333	

For successful runs, the deficit values and stress excess are zero. The objective value is adjusted by the penalty factors whenever constraints are violated; otherwise, it matches the mass value, as it is the focus of our minimization function.

Random Sampling Comparison

To further verify our results, we compared the optimization performance and efficiency against a random sampling (R.S.) algorithm. Random sampling optimization generates a set of candidate solutions within specified boundary conditions. It is particularly useful in solving complex problems, including black-box functions. However, its main drawbacks are the lack of convergence guarantees to a global optimum and its relatively low computational efficiency.

The random sampling setup uses the same design variables, constraints, and objective function as our Bayesian optimization (B.O.) algorithm. It also calls Abaqus to compute the stress concentration at each iteration. The key difference is that, unlike the B.O. algorithm, our penalty system was not applied to influence the calculation of the objective values. Here is a plot showcasing the results of running fifty iterations of both algorithms:

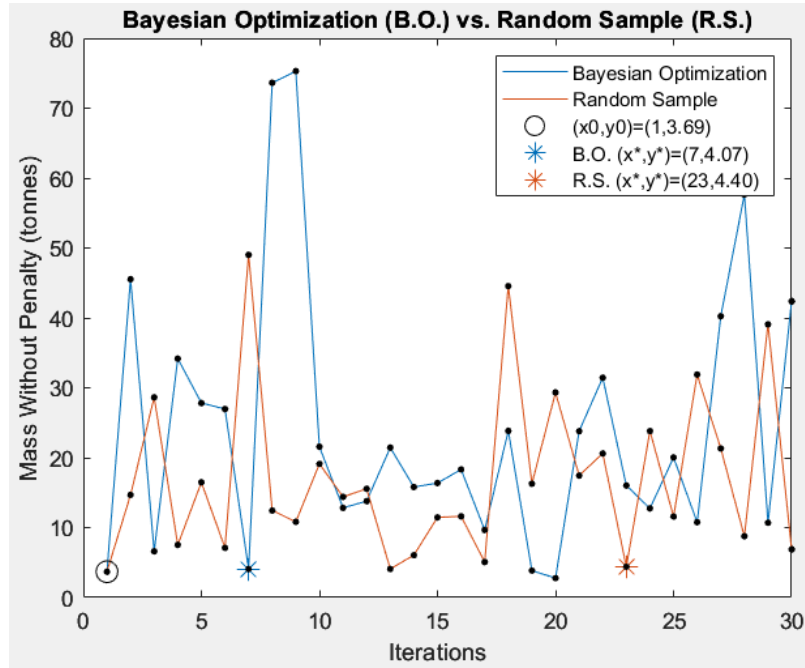


Figure 9. Plot comparing R.S. and B.O. algorithms

The results indicate that our B.O. algorithm reaches a local minimum (while also meeting the safety factor conditions) more quickly than the R.S. algorithm. Although R.S. is a stochastic model and theoretically could find a better optimum faster than B.O., it lacks the probabilistic structure that would guarantee consistent performance. In running this test, we've verified that our chosen algorithm is, at the very least, a resource efficient method to finding local minimums.

Validation

Reality Check

To validate our results, we attempted to compare them with existing pressure vessel models that have similar designs and parameters (if applicable).

Volumetric Validation

Based on [8], the horizontal pressure vessel model G-240 J with a 16” outer diameter and 48” length has a volumetric capacity of 55 gallons. Converting to metric units:

16 inches = 406.4 mm (diameter) = 203.2 mm (radius)

48 inches = 1219.2 mm (length)

55 gallons = 2.082e+8 mm³ (internal volume)

For our design, the required internal volume is:

$$V_{\text{req}} = 2.271\text{e}+8 \text{ mm}^3 = 60 \text{ gallons}$$

Computational Check: $L = 1200$, $R = 300$, $t_s = 180.678$, $t_h = 69.308$, $t_n = 100$ (all in mm)

Note: Our thickness is extruded from the radial centerline, so the equation for inner radius is:

$R_{\text{inner}} = R - t$ (for shell, endcap, and nozzle of different thicknesses)

$$V_{\text{internal}} = L\pi\left(R - \frac{t_s}{2}\right)^2 + \frac{4}{3}\pi\left(R - \frac{t_h}{2}\right)^3 + 150\pi\left(R - \frac{t_n}{2}\right)^2 \quad (12)$$

$$V_{\text{internal}} = 2.73\text{e}+8 \text{ mm}^3 \approx 72 \text{ gallons}, V_{\text{internal}} = 2.44\text{e}+8 \text{ mm}^3 \approx 64.5 \text{ gallons (without nozzle)}$$

Since the real-life model doesn't have its thickness and nozzle dimensions stated, we believe that our design is volumetrically realistic, considering that our radius is 3/2 the size of theirs.

Mass Validation

To validate our mass results, we compared them with pressure vessels subjected to the same internal pressure of 10 MPa. Specifically, a research paper [9] on vessel design for nuclear facilities provided data closely aligned with our material density and part thicknesses. While their model is similar in shape to ours, it does not include nozzles.

Design Parameters of the research paper:

- Factor of Safety: 2
- Uniform Internal Pressure: 10 MPa
- Uniform Shell Thickness: 115 mm
- Radius: 204 mm
- Length: 1219 mm
- Material Density (stainless steel): 7.5×10^{-9} tonnes/mm³

Using Eq.3 to approximate the mass of their design (omitting the nozzle and penalty functions):

$$M = \rho(2\pi LRt + 4\pi R^2 t) = 2.194 \text{ tonnes} = 2194 \text{ kg}$$

Our optimal mass is 3.785 tonnes (3785 kg), which is slightly less than double their design's approximated mass. However, considering that our radius is 100 mm greater and our factor of safety is 3, it is reasonable to assume that our mass is realistic when compared to existing designs.

Compared to typical industrial pressure vessels, gas cylinders with a high pressure of 15 MPa weigh around 36 kg for a 1.5 m long, 0.25 m diameter vessel, while industrial horizontal vessels range from 3 to 10 tonnes (for models of similar sizes to our design). Given that our optimized vessel operates at 10 MPa and has a structurally feasible thickness-to-diameter ratio, the resulting mass of 3.79 tonnes aligns with real-world expectations.

K-Fold Cross Validation

In the context of B.O., K-Fold cross-validation can be used to assess how well the algorithm is exploring different regions of the solution space. The silhouette score serves as a metric to evaluate how well-separated the points sampled during optimization are. A high silhouette score indicates that our algorithm is effectively exploring different, distinct regions of the search space, meaning the samples are well-separated and not too similar to each other. This suggests that our algorithm is good at distinguishing between configurations that lead to good versus poor outcomes, helping it navigate the solution space efficiently.

If the silhouette score is high, it suggests that our design variables (e.g., length and radius) are being chosen from diverse, well-defined regions, which could explain why certain values, like the lower bounds of length and radius, are being selected more often. This might indicate that those bounds are especially important or optimal for our design.

Silhouette scores range from -1 to 1: negative values indicate poor clustering, positive values indicate better clustering, and a score of zero indicates ambiguity. For example, if we assign our mass data into three separate clusters (one for each: high, medium, and low masses) and calculate a silhouette score, having a high score would mean that there is a decent separation of data (i.e. the optimization algorithm is capturing a good amount high and low masses, which is great to use as the sampling data for EI, but not indicative of convergence to low masses).

Table 4: K-Fold Values

Number of Folds	Number of Clusters	Silhouette Score
2	3	0.685
5	3	0.639
10	3	0.628

(Note: the silhouette scores above were found using *masswithoutpenalty* data from 60_x1.csv)

Conclusion & Future Plans

Conclusion

The project successfully developed and implemented an optimization framework for designing a pressure vessel with minimal mass while maintaining structural integrity and satisfying all constraints. Using Bayesian Optimization coupled with Abaqus Finite Element Analysis (FEA), we systematically explored the design space and identified optimal configurations that balance material usage, performance, and regulatory compliance.

The key findings from the study include: The optimization algorithm efficiently minimized mass while satisfying stress and geometric constraints, demonstrating the suitability of Bayesian Optimization for pressure vessel design. However, different initial conditions led to slightly different optimal results due to the exploration-exploitation tradeoff inherent in Bayesian Optimization. Running 10 vs. 30 optimization iterations showed that fewer iterations still resulted in reliable convergence, demonstrating computational efficiency.

Moreover, a penalty function-based constraint handling method was implemented to ensure continuous optimization, eliminating the need for strict infeasibility filtering. The optimization results indicated that the methodology effectively guides the design process toward an efficient and manufacturable solution.

Table 5: Optimal Values for Each Test

Starting Point	R (mm)	L (mm)	t_s (mm)	t_h (mm)	t_n (mm)	Objective	Mass (tonnes)
X1	300	1200	180.678	69.308	100	3.785	3.785
X2	340.748	2607.32	166.827	61.506	76.475	7.905	7.905
X3	300	3007.65	106.963	300	100	7.380	7.380
Random Sampling	305.806	2230.14	196.104	43.478	62.284	6.869	6.869
X1 (100)	311.38	1200	109.512	300	94.7436	4.869	4.869

The optimal values for each case are summarized in Table 5, which show that the first starting point has the lowest mass and objective value. We can also make sense of the optimal design variable as the shell thickness should be the greatest of the thicknesses as the maximum stresses always occur at this geometry.

Future Work

Future work will focus on fine-tuning the penalty scalars for each constraint to ensure that the objective values remain on the same scale as the masses, while still being recognized by the optimization algorithm as infeasible when violated. This process requires running the algorithm multiple times and adjusting one scalar at a time, which is both tedious and time-consuming. To improve efficiency, we plan to implement error statements that specify the iteration step and the breached constraint during an extensive run. We would then adjust the penalties in order of their iteration, starting with the lowest, to minimize the number of iterations we'll have to call.

In addition to adjusting the penalties, we could explore modifying the boundary conditions in our Abaqus model. If successful, this would provide new lower bounds for the design variables, expanding the solution space and potentially leading to lower minimum masses that align with real-world expectations. If resetting the boundaries isn't feasible, the next best approach would be to reposition the nozzle along the y-axis to prevent extrusion failure when the shell length is too short. If these options don't work, we could recreate the model in Abaqus, scaling the part down from the start to ensure smaller, acceptable values while preserving the key elements of our design.

Achieved Learning

Alexander Torres-Soto

I learned about how to formulate a sizing problem based on geometric parameters. The project helped me gain this skill that will later help me in design roles and will eliminate some time of my learning curve when entering industry. I also learned how to set up and use Abaqus as an FEA solver, this is especially important as now I understand the capabilities of writing Python script for Abaqus.

In terms of optimization, I learned how to couple Abaqus python scripts with python optimization functions and create continuous penalty functions that incorporate constraint violations. This is extremely important as we can now set up constraints that could not be captured in FEA optimizers.

Xing Yu Chen

In terms of performance and attained ability, I learned the basics of navigating the Abaqus terminal and design space. I learned how to call Abaqus through a Python script and modify the model parameters to align with my design specifications (design variables). I gained experience retrieving key Abaqus output data and storing it in a file after each iteration. I also learned how to structure the basic framework for the gp_minimize algorithm, ensuring it writes an input file and reads the Abaqus output file during each operation. Additionally, I implemented the random sampling algorithm and visualized the results using Plot_Maker.m, interpreting the trends within the data. I learned how to validate the effectiveness of our algorithm's search process through k-fold cross-validation.

In the context of our project, I gained foundational knowledge in designing pressure vessels and learned that they have a wide range of applications. I also explored how to apply Bayesian optimization to solve optimization problems involving black-box functions. I figured out how to validate our results by comparing them with numerical solvers and other optimization algorithms. Most importantly, this experience has given me a deeper understanding of how to approach complex optimization problems in future projects.

References

- [1] M. Mokhtarian, M. Shahriari, and S. Najafi, "Optimization of pressure vessel design using computational techniques," *International Journal of Pressure Vessels and Piping*, vol. 188, p. 104199, 2020.
- [2] X. Zhu, Y. Chen, and H. Wang, "Structural optimization of cylindrical pressure vessels under internal pressure," *Journal of Mechanical Engineering Science*, vol. 233, no. 11, pp. 3421-3435, 2019.
- [3] J. Xie, Z. Li, and R. Huang, "Bayesian optimization methods for engineering design applications," *Structural and Multidisciplinary Optimization*, vol. 64, no. 3, pp. 987-1003, 2021.
- [4] "SA 516 steel plates grade 60, 65 & 70," American Alloy Steel, <https://www.aasteel.com/sa-516/>
- [5] Metals Handbook, Vol.1 - Properties and Selection: Irons, Steels, and High-Performance Alloys, ASM International 10th Ed.1990.
https://www.matweb.com/search/datasheet_print.aspx?matguid=9ccee2d0841a404ca504620085056e14
- [6] "Thick walled cylinders," Thick Walled Cylinders,
<https://courses.washington.edu/me354a/Thick%20Walled%20Cylinders.pdf>
- [7] "ASME Boiler and Pressure Vessel Code" The American Society of Mechanical Engineers, New York, Jul. 1, 2023
- [8] "Standard Horizontal & vertical pressure vessels," Central Fabricators,
<https://centralfabricators.com/standard-pressure-vessels/>
- [9] Author links open overlay panelS. Arunkumar et al., "Design optimization of horizontal pressure vessel," *Materials Today: Proceedings*,
<https://www.sciencedirect.com/science/article/pii/S2214785320310695?via%3Dihub>
- [10] Person and Savree.com, "Pressurized water reactor (PWR) vessel explained," saVRee,
<https://www.savree.com/en/encyclopedia/pressurized-water-reactor-pwr-vessel>
- [11] J. Brownlee, "A gentle introduction to k-fold cross-validation,"
MachineLearningMastery.com, <https://machinelearningmastery.com/k-fold-cross-validation/>

Appendix

All Code Files Uploaded Here: https://github.com/XYChen10/ME341_Project_Code

- Run_Abaqus.py - creates Abaqus model by reading inputs from optimization output file
- Bayesian_Optimization.py - runs the B.O. algorithm and calls Run_Abaqus.py
- Random_Sample.py - runs random sample optimization and calls Run_Abaqus.py
- Plot_Maker.py - plots our optimization results (on python)
- Plot_Maker.m - plots our optimization results (on Matlab)
- Abaqus_Figure_Maker.py - used to make FEA figures from our results
- README.txt - instructions for each file
- K-Fold_Cross_Validation.py - used for K-FCV on data